

Optimizing Assets for Authoring and Consumption in USD

Harmony M. Li
Walt Disney Animation Studios
Burbank, USA
harmony.li@disney.com

Ricky Rieckenberg
Walt Disney Animation Studios
Burbank, USA
ricky.rieckenberg@disney.com

Neelima Karanam
Walt Disney Animation Studios
Burbank, USA
neelima.karanam@disney.com

Emily Vo
Walt Disney Animation Studios
Burbank, USA
emily.h.vo@disney.com

Kelsey Hurley
Walt Disney Animation Studios
Burbank, USA
kelsey.hurley@disney.com

ABSTRACT

Walt Disney Animation Studios has used Universal Scene Description (USD) [Pixar 2016] as the backbone of its production pipeline since "Encanto" in 2021 [Miller et al. 2022]. In this talk, we introduce a new asset structure that addresses speed issues from our initial asset structure design and vastly simplifies asset authorship. The new asset structure helped to (1) streamline and decouple asset authorship and shot consumption, (2) enable new authoring workflows that better take advantage of USD's multi-stage model and (3) open the door for shot focused asset-based optimizations.

ACM Reference Format:

Harmony M. Li, Ricky Rieckenberg, Neelima Karanam, Emily Vo, and Kelsey Hurley. 2024. Optimizing Assets for Authoring and Consumption in USD. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Talks (SIGGRAPH Talks '24)*, July 27–August 01, 2024, Denver, CO, USA. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3641233.3664335>

1 INTRODUCTION

Our studio's asset workflow is highly dependent on the ability to create many asset variants that have combinations of inherited components from one another. A *component* refers to a logical unit of the asset (e.g. a groom, rig, or geometry); an *asset variant* refers to a usable combination of components. For example, a typical character will have a main asset variant that contains the base material, geometry, etc.; sequence specific asset variants (e.g. wet variant, costume changes) are created as inherited variants from the main variant, where deltas from the main asset variant are authored into the inherited asset variant. When changes are made to the main asset variant, inherited asset variants will also pick up changes from the main asset variant automatically.

A primary goal of developing a USD-native pipeline was to take advantage of USD's composition constructs to better model asset and shot composition [Miller et al. 2022]; however, our initial asset structure was built around department workflows and encapsulated asset variant inheritance as inherits arcs from class prims. While

this structure made it straightforward to directly author asset data to a single layer, this structure also (1) lacked compositional transparency, (2) slowed down authoring workflows, and (3) slowed shot load times. Some speed issues were addressed by changing payload arcs to references, but encapsulation of the payload arc caused composition issues, and workarounds to represent destructive edits were brittle [Vo et al. 2023].

2 ASSET STRUCTURE

While detailed asset authorship information (e.g. internal asset composition) is useful for debugging, most of this information is not directly necessary for shot production. Thus, we can conceptualize an asset's data as two related but decoupled structures:

- *source structure*, an authoring-optimized structure that contains all composition information necessary for understanding how changes are propagated amongst inherited asset variants, and
- *compiled structure*, a shot-optimized structure that is compiled from the authoring optimized structure but is stripped of any unnecessary internal composition that may weigh down the shot.

The primary goal of the source structure is to clearly depict how an asset is made, both in data and in composition, so that the asset can be maintained and propagated; in contrast, the goal of the compiled structure is to be as minimal and optimized for shots as possible.

2.1 Source Structure

In the new asset source structure, each component is a layer stack, as shown in Figure 1. Like the previous asset structure, each asset department authors to its own component delta, including payloading data-heavy subcomponents like geometry, to its own layer in the component layer stack. Each asset variant is a layer stack of component stack root layers that comprise the variant. Each asset is a collection of variants in a variant set that each reference their respective asset variant stack root layer.

Asset variant inheritance is represented at the component level : if a component inherits from another asset variant's component, the parent's component stack root layer is sublayered as the lowest layer in the inheriting component's layer stack. Doing so allows for each component to have its own inheritance tree (e.g. material inheritance vs geometric inheritance). The nested sublayer structure allows for easy parsing of inheritance and department contributions while maintaining straightforward data authoring.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
SIGGRAPH Talks '24, July 27–August 01, 2024, Denver, CO, USA
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0515-1/24/07
<https://doi.org/10.1145/3641233.3664335>

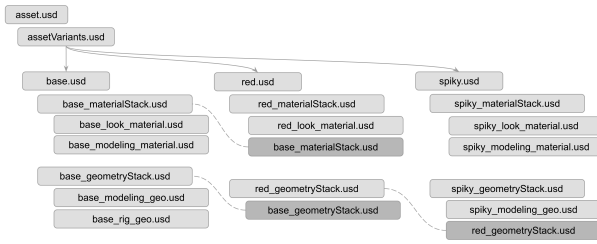


Figure 1: Illustration of Asset Source Structure of an asset with three asset variants : base, red and spiky. The asset is rooted at `asset.usd` and sublayers in `assetVariants.usd`, which introduces the three asset variants via a variant set that swaps which asset variant stack to which it refers. Asset departments author to their own component layers in the material and geometry component stacks. The red asset variant inherits both material and geometry from base. As such, red’s material stack includes base’s materialStack.usd as its lowest layer, and authors look material changes to red’s look material layer. Since red does not include a delta in geometry, it simply inherits base’s geometry stack. In contrast, the spiky only inherits its geometry. If spiky had originally inherited its materials from red’s material and no longer wanted to do so, spiky’s look material would be a flattened copy of red’s materials. © 2024 Disney

2.2 Compiled Structure

The source structure’s sublayering mechanism mainly tracks asset variant inheritance and asset department contribution, which is unnecessary for shot consumption. The compilation process for the compiled structure flattens each of the component layer stacks. Components that are heavy are payloaded in, while lighter components are flattened directly into the asset variant stack layer. The resulting compiled structure is a collection of variants in a variant set, but each variant is now pointing to a flattened structure with all intermediary component composition arcs baked in. Each asset is compiled during the publish process as the asset is released for shot consumption. All of the source asset data files, along with the compiled files, are versioned on publish as a way to tag associated compiled structure files with their sources. Shots directly consume the installed compiled structure from the release asset area, and are treated as a compiled product of the original source asset data files.

3 WORKFLOW CHANGES

3.1 Authoring Workflows

Asset authoring tools read each asset variant layer stack as their own stage, allowing for multiple asset variants of an asset to be open at the same time. When interactive changes are made on any parent asset variant layers, USD multistage composition automatically propagates the changes from parent to all open child asset variant stages to another in the same authoring session.

To avoid confusion over which layer tools should access and edit, we created an expandable API to (1) encapsulate the source layer structure and automatically read and/or create necessary layers and sublayers on creation and deletion of asset variants and components, (2) redirect edits to the correct layers via plugins that know about pipeline and DCC assumptions, and (3) clean up orphaned deltas and data across layer stacks on destructive edits (e.g. renames).

3.2 Debugging Workflows

Decoupling the asset and shot structures is a large change to the studio. To ease adoption, we created tooling that wrapped the compilation process to allow for incremental compilation. Each run could take either published or unpublished data, and optionally republish the compiled asset. The compilation process outputs the compiled asset structure to a temporary location on disk before publish to help hands-on debugging. On asset release, the compiled structure is copied and installed into the release asset area.

4 ASSET-BASED SHOT OPTIMIZATIONS

The build process from source structure to compiled structure effectively decouples the two structures and offers a unique opportunity to transform the compiled structure’s USD data to better suit mass consumption in shots and shot authoring. This includes, but is not limited to, processing and building cache structures into the asset info and metadata, and restructuring compiled asset data to take advantage of different USD constructs. For example, we currently cache each asset’s variant inheritance in metadata in the build process so that shot tooling can access the inheritance information without composing the asset. Similarly, we reorganize our asset data to turn on USD scenegraph instancing on all of our elements to take advantage of scenegraph reuse between assets.

5 DISCUSSION AND FUTURE WORK

Our new asset structure was deployed during the development of Disney’s “Wish”. The asset structure changes unlocked workflows that allow artists to edit asset properties on a parent variant while viewing and rendering a child variant, enabling quick verification of sweeping changes. Decoupling of the asset and shot structures also made subsequent asset structure changes much easier; previous asset structure changes would require all artists to pause work so that all assets and consuming shots could receive the changes in lockstep. With the new decoupled structure, asset structure changes affecting only released assets could be deployed as a complete recompile of all of the show’s assets from published source asset structure without requiring artists to stop. Since the asset compilation tooling could be run on the renderfarm, show-wide asset recompilation could be run in parallel in a matter of minutes. Much of later asset optimizations and mass asset fixes were deployed this way on “Wish”, which reduced artist down time.

In the future, we would like to further break larger components into smaller logical ones to allow for better reuse across asset variants and among assets. Moving forward, we hope to revisit workflows (such as grooming and hair) to better take advantage of the multi-stage variant model.

REFERENCES

- Tad Miller, Harmony Li, Neelima Karanam, Nadim Sinno, and Todd Scopio. 2022. Making Encanto with USD: Rebuilding a Production Pipeline Working from Home. In *ACM SIGGRAPH 2022 Talks*. Article 12, 2 pages. <https://doi.org/10.1145/3532836.3536236>
- Pixar. 2016. Introduction to USD. <http://openusd.org/>.
- Emily Vo, George Rieckenberg, and Ernest Petti. 2023. Honing USD: Lessons Learned and Workflow Enhancements at Walt Disney Animation Studios. In *ACM SIGGRAPH 2023 Talks*. Article 13, 2 pages. <https://doi.org/10.1145/3587421.3595421>